COLLEGE OF NATURAL AND COMPUTATIONAL SCIENCE

SCHOOL OF INFORMATION SCIENCE

SMART AGRICULTURAL MARKETPLACE PLATFORM

MOBILE COMPUTING PROJECT DOCUMENTATION

Group Members	**ID No**
1. Dawit Yohans	UGR/4173/16
2. Kassa Gebrekidan	UGR/4773/16
3. Chekole Ngusalem	UGR/7086/16
4. Gebre Kahsay	UGR/2154/16
5. Tsegay Assefa	UGR/3204/16
6. Temesgen Abdissa	UGR/8666/13

Submitted To: Instructor G/Micheal

Submission Date: June 22, 2026

TABLE OF CONTENT

AGRISPARK MOBILE APP DOCUMENTATION

1. Introduction

Agri Spark is a mobile application built with React Native that connects agricultural producers (farmers) with bulk buyers such as hotels, companies, wholesalers, and retailers. The platform creates a digital marketplace where farmers can directly list and sell their agricultural products while buyers can easily discover, order, and communicate with suppliers in real time.

The system enables:

- Direct product listing by farmers
- Bulk purchasing by buyers
- Real-time order management
- Efficient communication between supply and demand
- Product discovery and marketplace browsing
- Secure and role-based user access
- Real-time chat between buyers and farmers

Agri Spark is designed to simplify agricultural commerce by reducing unnecessary intermediaries, improving market access for farmers, and creating a more efficient supply chain between producers and buyers.

2. Problem Description

In many parts of Ethiopia, especially rural farming communities, farmers face serious challenges in selling their agricultural products at the right time and price. During harvesting seasons, farmers often produce large quantities of crops such as cabbage, onions, tomatoes, potatoes, and other vegetables. However, because of limited market access, poor communication with buyers, and lack of digital trading systems, many farmers are unable to sell their products before they spoil.

As a result:

- Products are wasted or spoiled
- Farmers experience financial losses
- Prices become unstable during high production seasons
- Farmers are forced to sell products at very low prices
- In some cases, unsold vegetables are even given to cattle because they cannot reach buyers in time

At the same time, many hotels, restaurants, wholesalers, and companies struggle to find reliable suppliers quickly and efficiently.

Agri Spark aims to solve this problem by providing a centralized digital marketplace where farmers and buyers can connect directly. Farmers can advertise available products, manage stock, receive orders, and communicate with buyers instantly, while buyers can browse products, place orders, and coordinate deliveries in one platform.

The platform helps:

- Reduce agricultural product waste
- Increase farmers' market opportunities
- Improve communication between producers and buyers
- Support fair and transparent trading
- Modernize agricultural commerce using mobile technology

3. Project Goal

The goal of AgriSpark is to create a reliable, scalable, and user-friendly agricultural marketplace platform that empowers Ethiopian farmers through digital access to markets while helping buyers efficiently source fresh agricultural products

.

4. Objectives

General Objective

Develop a mobile platform that digitizes agricultural commerce.

Specific Objectives

- Enable farmers to list products.
- Enable buyers to browse products.
- Facilitate direct communication.
- Support order management.
- Provide multilingual support.
- Deliver AI-assisted guidance.
- Reduce agricultural product waste.

5. AgriSpark – Requirements Specification

5.1. User Roles & Functional Requirements

➢ **Farmer (Supplier)**

Authentication

Description:

Allows farmers to securely create accounts and log in with role farmers.

Functional Requirements:

- Farmer must register using:
    - Name
    - Phone/email
    - Password
    - Location
- System must validate input fields
- Farmer must log in using credentials
- System must maintain session (user stays logged in)

- **Product Management**

Description:

Farmers can create and manage their agricultural product listings.

Functional Requirements:

- Farmer can:
  - Add product ○ Edit product ○ Delete product ● Product must include: ○ Product name
  - ○ Category (e.g., grains, vegetables)
  - ○ Price per unit
  - ○ Quantity available
  - ○ Location
  - ○ Image
- System must store product data in database
- System must associate product with the farmer

- **Product Viewing (Own Products)**

Description:

Farmers can see all products they have listed.

Functional Requirements:

✓ Display list of farmer's products

✓ Show:

- ✧ Name
- ✧ Price
- ✧ Quantity

✓ Allow update/delete actions

- **Order Management**

Description:

Farmers manage incoming orders from buyers.

Functional Requirements:

- Farmer can:

✓ View all orders for their products

●Each order must display:

- Buyer name
- Product
- Quantity ordered
- Status

- Farmer can:
  - Accept order
  - Reject order

➢ **Buyer**

Authentication

Description

The authentication system enables buyers (such as hotels, companies, and wholesalers) to securely create accounts and access the AgriSpark platform. It ensures that only authorized users can browse products and place orders.

Functional Requirements

The buyer must be able to **register an account** by providing:

- ✓ Full name or company name
- ✓ Phone number or email
- ✓ Password
- ✓ Location (city/region)

✧ The system must:

✓ Validate all input fields (e.g., no empty fields, valid email/phone format)

✓ Ensure that each email/phone number is **unique**

✓ Store user credentials securely (e.g., hashed password)

✓ The buyer must be able to **log in** using:

- ➢ Email/phone
- ➢ Password

✧ The system must:

✓ Verify login credentials

- ✓ Display an error message for invalid login attempts
- ✓ Allow access only if credentials are correct

- ✧ The system must assign the user role as:"buyer"
- ✧ The system must maintain a **user session**:
- ✓ Keep the buyer logged in until they log out
- ✓ Allow automatic login if session is still valid

The buyer must be able to **log out** of the system securely

- ➢ **Validation Rules**
  - Password must have a minimum length (e.g., 6 characters)
  - Email must follow a valid format (if used)
  - Phone number must follow a valid format
  - Duplicate accounts are not allowed **Error Handling**
  - If registration fails:
  - Show clear error messages (e.g., “Email already exists”)
  - If login fails:
    - ✓ Show “Invalid credentials”
- ➢ **Product Browsing**

Description:

Buyers can explore available agricultural products.

Functional Requirements: ● System must display all available products ● Each product shows:

- ○ Name
- ○ Price
- ○ Quantity
- ○ Location

- Buyer can scroll list
- (Optional) Filter by:

- Category
- Price
- Location

➢ **Product Details**

Description:

Detailed view of a selected product.

Functional Requirements:

- Show:
  - Product image
  - Full description
  - Farmer info
  - Price
  - Available quantity

➢ **Order Placement**

Description:

Buyers can place bulk orders.

Functional Requirements:

● Buyer selects quantity

● System validates:

✓ Quantity ≤ available stock

✓ Buyer submits order

✓ System creates order record

● Order status = “Pending”

➢ **Admin Dashboard Description**

The **Admin** role manages the platform, monitors activities, and ensures that both farmers and buyers follow the rules. The admin has full access to all users and can resolve disputes.

Functional Requirements

- **User Management**
  - View all registered users (farmers and buyers)
  - Approve new farmers (optional)
  - Deactivate or delete suspicious users
  - Edit user information if needed
- **Product Monitoring**
  - View all products listed by farmers
  - Remove or flag inappropriate products

● **Order Monitoring**

- View all orders in the system
- Track order statuses
- Resolve disputes between farmers and buyers

● **Analytics & Reports** (Optional advanced feature)

- Number of active farmers/buyers
- Number of products listed
- Top-selling products
- Order trends over time

● **Notifications**

- Send system-wide announcements to farmers or buyers

Access Rules

- Only users with role = "admin" can access the dashboard
- Admin cannot place orders or list products
- Admin can view all system activity

➢ Core System Features

Authentication System

Description:

Handles user identity and access control.

Requirements:

- Role-based access:
    - Farmer
    - Buyer
    - Admin
- Secure password storage (hashing)
- Input validation
- Error handling:
    - Invalid login
    - Duplicate account

➢ Product Management System

Description:

Handles creation and maintenance of product listings.

Requirements:

- CRUD operations:
    - Create
    - Read
    - Update

    - Delete
- Each product linked to a farmer (foreign key)
- Real-time availability update (when order is placed)

➢ Product Browsing System

Description:

Allows buyers to explore available products.

Requirements:

- Fetch products from backend (via Express.js API)
- Display in list format
- Efficient loading (pagination optional)

➢ Order Management System

Description:

Handles all transactions between buyers and farmers.

Requirements:

- Create order:
    - Buyer ID
    - Product ID
    - Quantity
- Track status:
    - Pending
    - Accepted
    - Rejected
- Update product quantity after order acceptance
- Prevent over-ordering

➢ Define non-functional requirements

Security

- Password must be hashed

- JWT authentication required

Performance

- App should load within 3 seconds
- Support multiple users simultaneously

Usability

- Simple UI
- Easy navigation

Availability

- System available 24/7

Reliability

- No data loss on failure

6. Chat Between Farmer & Buyer

Description

The **chat system** enables real-time communication between buyers and farmers. It allows negotiation, order clarification, and relationship building.

Functional Requirements

- **Messaging**
    - ○Buyer can send a message to the farmer about a ordered products
    - ○ Farmer can reply to the buyer
- **Message Details**
    - ○ Timestamp
    - ○ Sender information (Farmer or Buyer)
    - ○ Message content (text, optional images in the future)
- **Notifications**
    - ○ Notify users of new messages

○ Optional push notifications (future enhancement)

● **Chat History**

○ Maintain conversation history

○Allow viewing previous messages for each product/order

Access Rules

- Only **buyer ↔ farmer** communication allowed
- Admin can **monitor chat** for moderation purposes (optional)
- Unauthorized users cannot access messages

7. AgriSpark AI Assistant

Description

Agri Spark AI is an intelligent chat-bot that assists farmers, buyers, and administrators in using the platform. It provides guidance, answers questions, and displays agricultural market information in multiple languages.

Functional Requirements

AI Chat Support

Users can:

- Ask how to use the application
- Get help with registration and login
- Learn how to add products, place orders, and manage accounts
- Receive answers to frequently asked questions **Market Price Information**

Users can:

- View standard market prices of agricultural products
- Browse products through a scrollable list
- Search products by name or category

Multi-Language Support

The AI Assistant shall support:

- English

- Amharic (አማርኛ)
- Afaan Oromo (Oromiffa)
- Tigrinya (ትግርኛ)

Users can select and switch languages at any time.

Farmer Assistance

- Product management guidance
- Market price information
- Order management support

Buyer Assistance

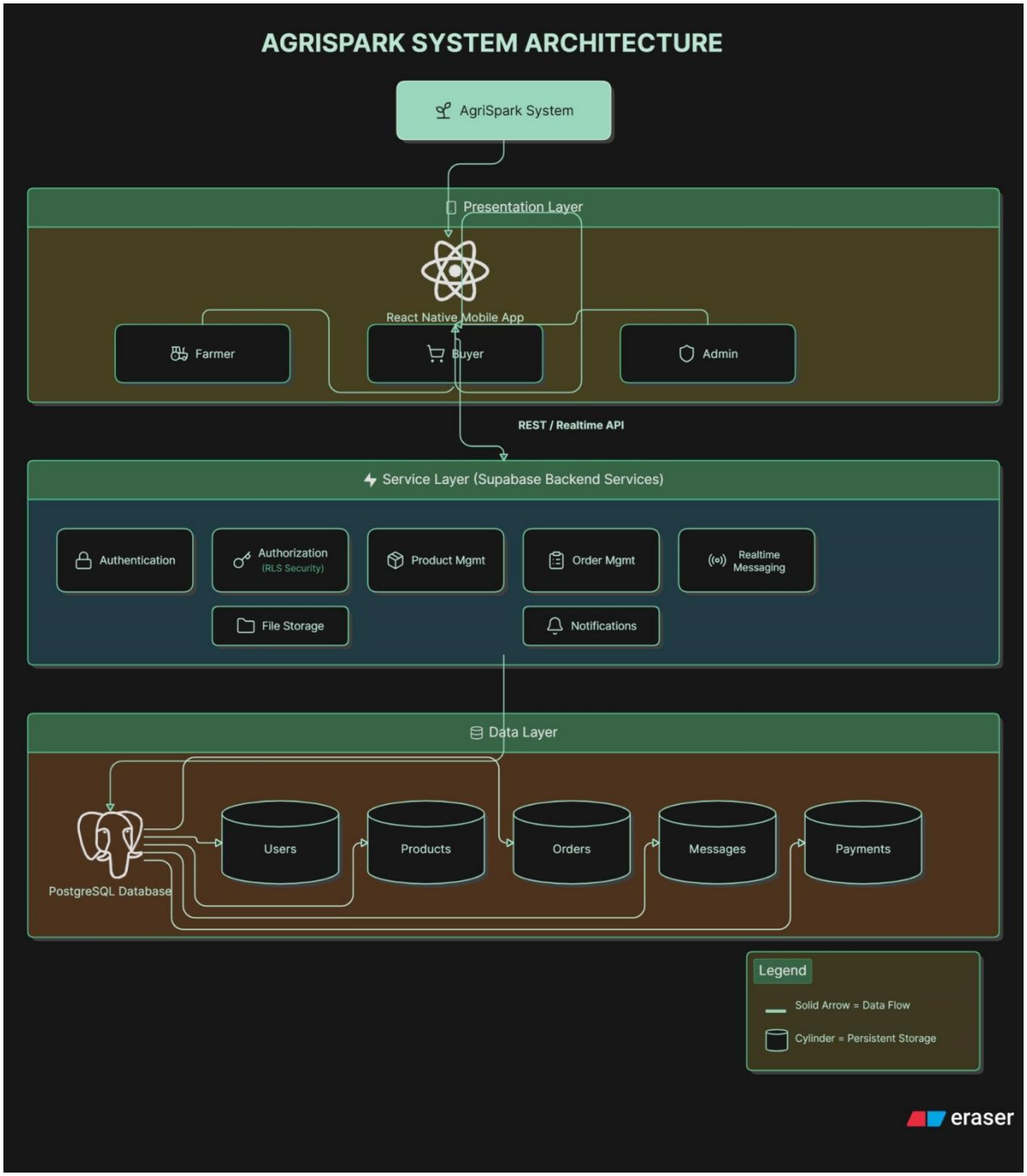
- Product search assistance
- Order and payment guidance
- Supplier communication support

Admin Assistance

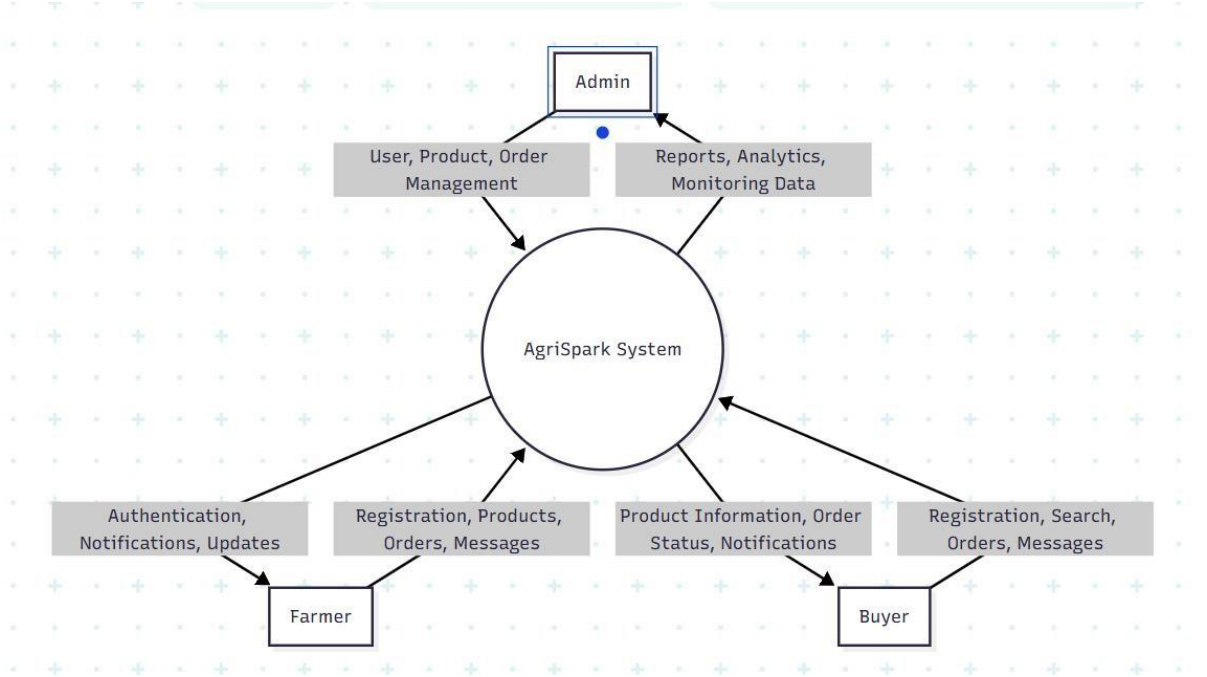
- Dashboard guidance
- User and product management support
- Report and system monitoring assistance

8. AgriSpark-Design

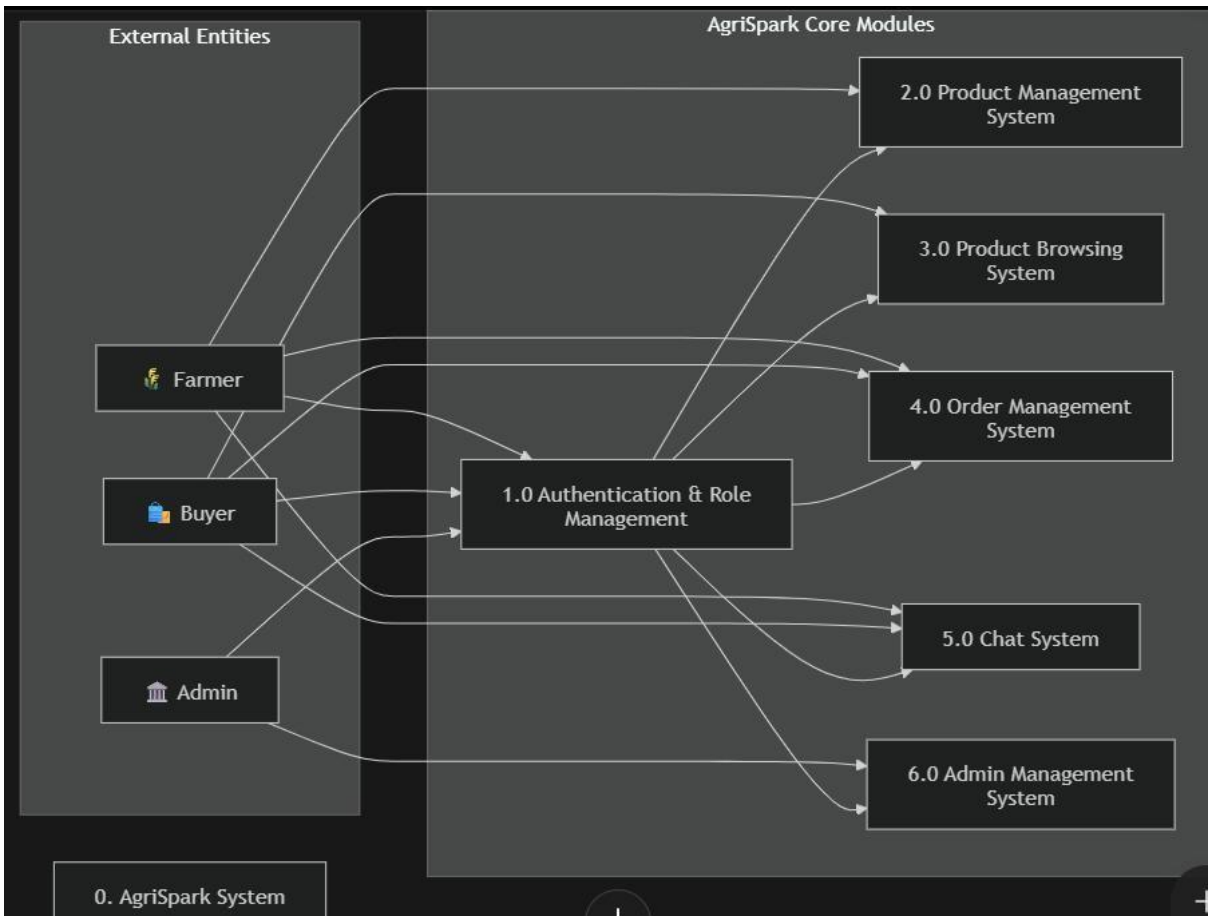
8.1.System Architectural design



8.2.Context Diagram:

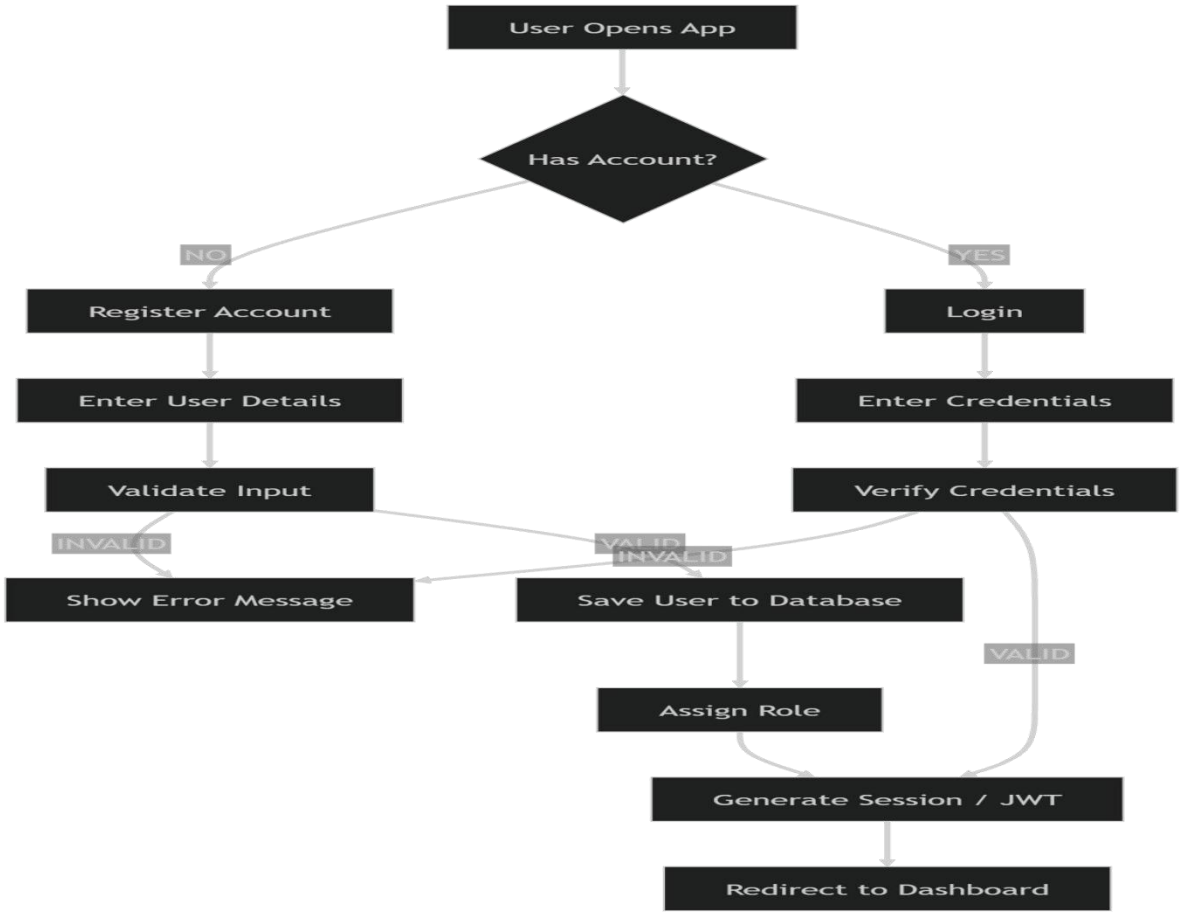


8.3.Level-0 DFD



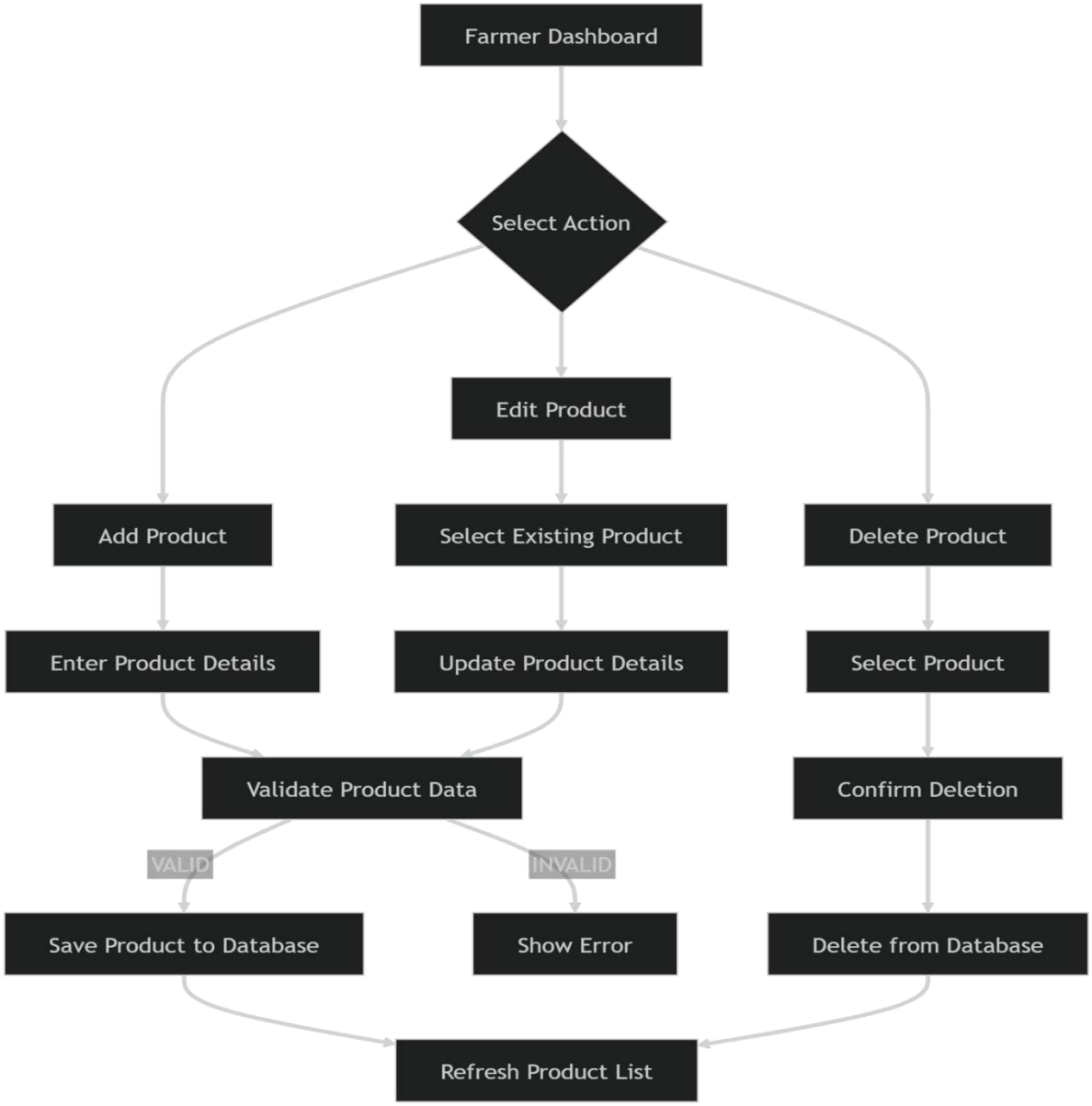
8.4. Authentication Flow

This flow shows how users enter the system. When the app opens, the user either registers or logs in. During registration, the system checks the input and creates an account if everything is valid. During login, the system verifies the credentials. If successful, a session (or JWT) is created and the user is redirected to their dashboard based on their role.



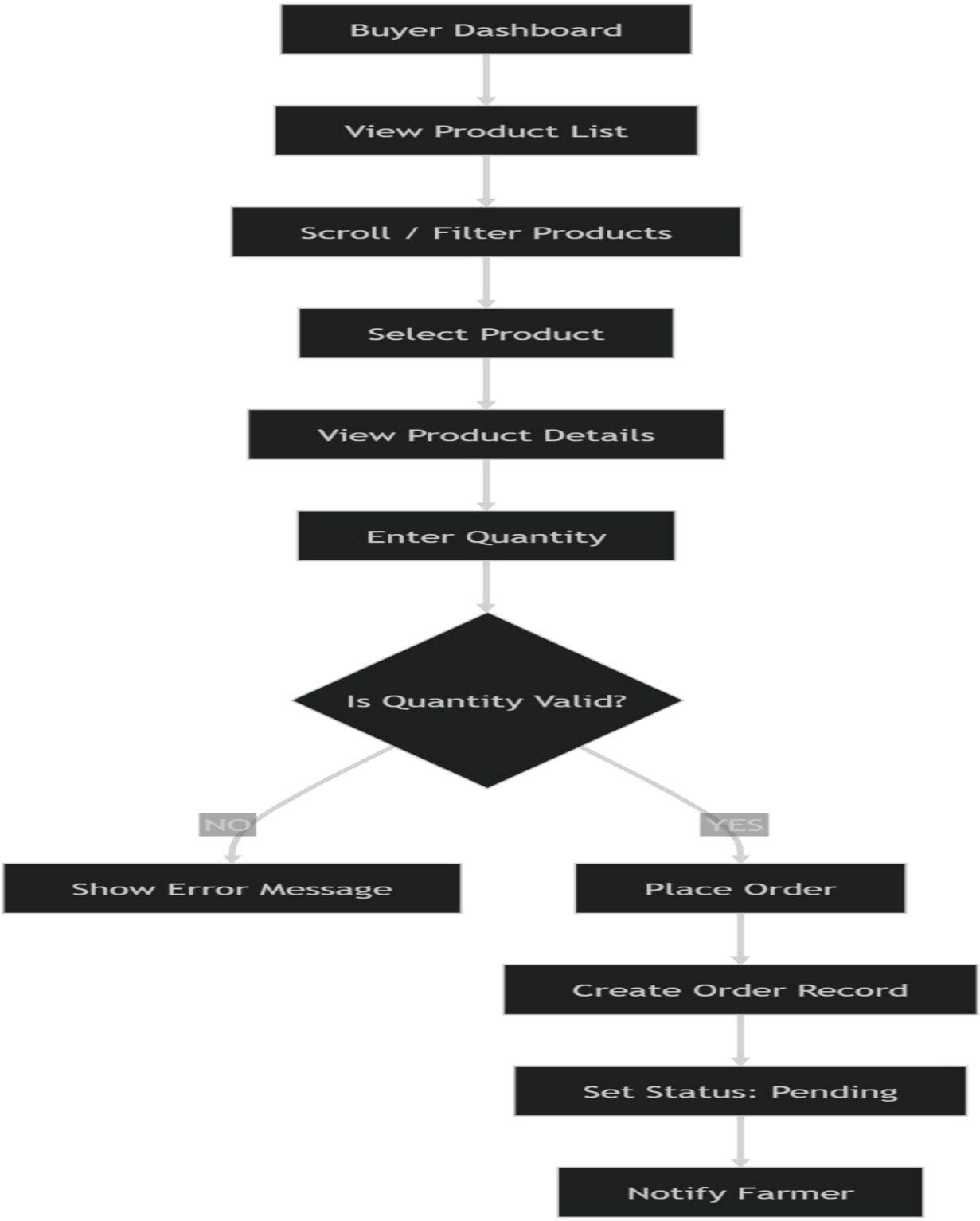
8.5. Farmer Product Management Flow

This flow explains how farmers manage their products. From the dashboard, a farmer can add, edit, or delete products. When adding or updating, the system validates the product details before saving them in the database. If deleting, the system asks for confirmation before removing the product. After any action, the product list is refreshed.



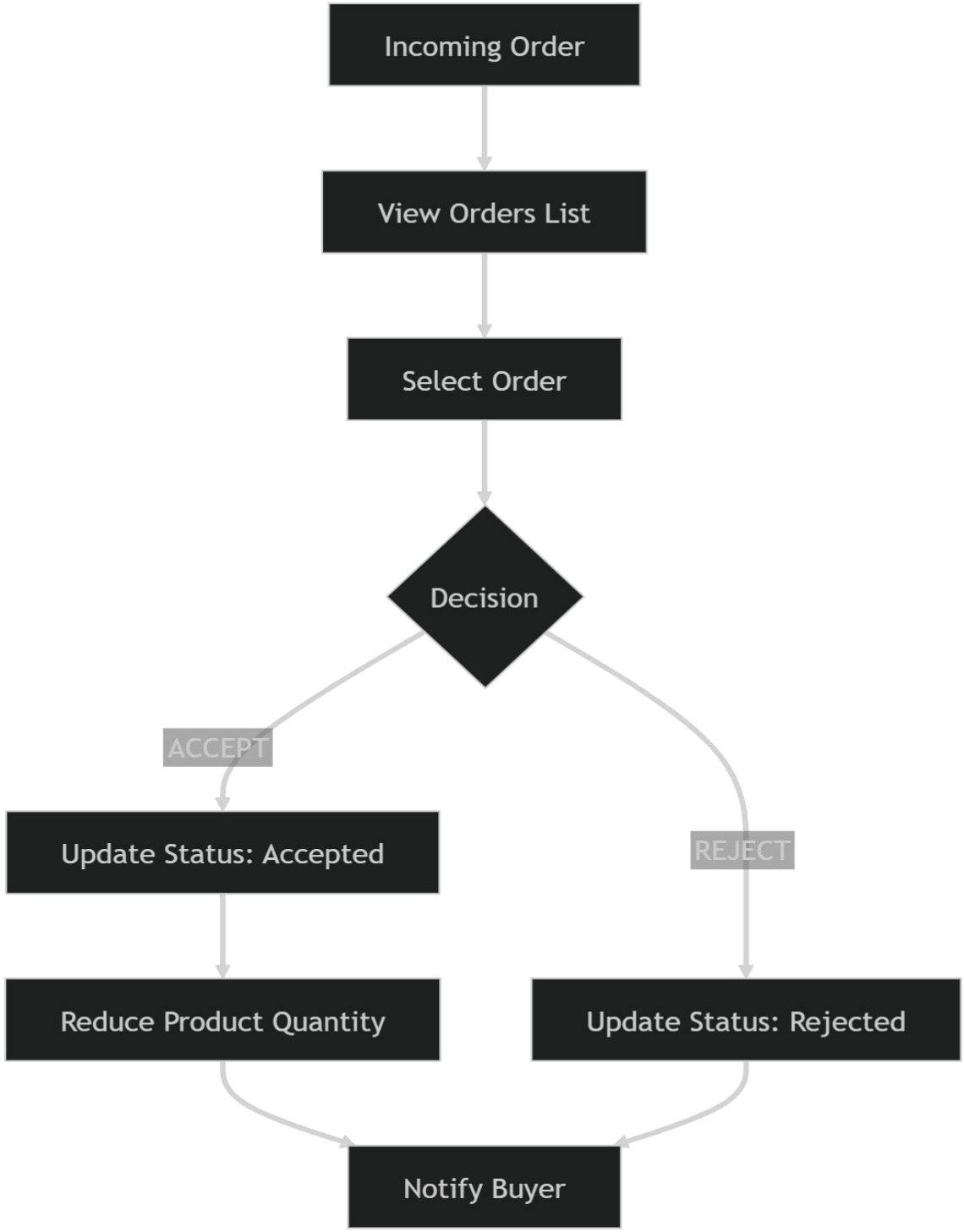
8.6. Buyer Browsing & Ordering Flow

This flow shows how buyers find and order products. The buyer browses available products, optionally filters them, and selects one to view details. After entering a quantity, the system checks if enough stock is available. If valid, the order is created with a “Pending” status and the farmer is notified.



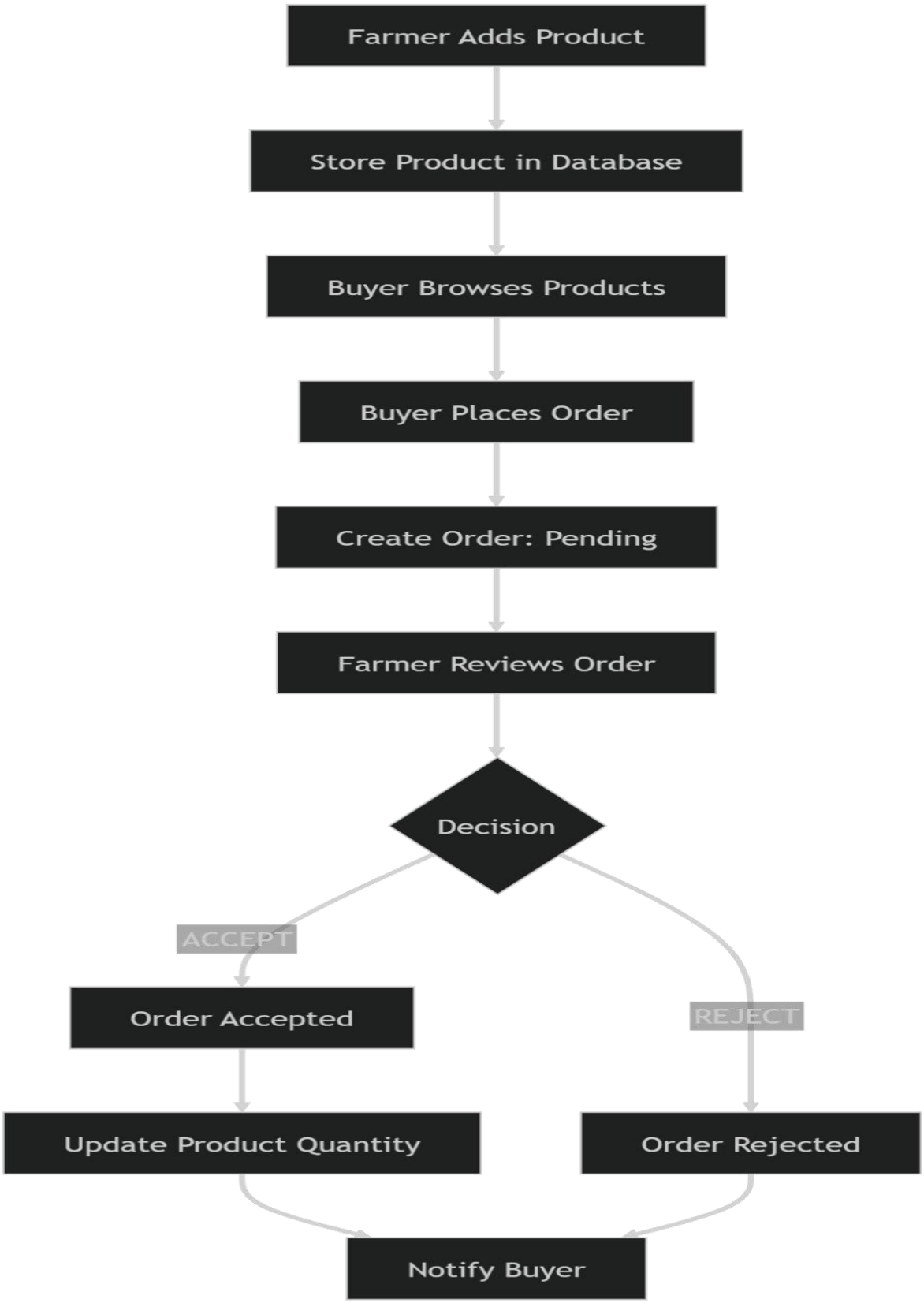
8.7. Order Management Flow (Farmer Side)

This flow describes how farmers handle incoming orders. The farmer views a list of orders and selects one to review. They then decide to accept or reject it. If accepted, the system updates the order status and reduces the product quantity. If rejected, the status is updated accordingly. In both cases, the buyer is notified.



8.8. Full System Flow (End-to-End)

This flow shows the complete interaction between farmer and buyer. A farmer adds a product, which becomes visible to buyers. A buyer places an order, which is marked as pending. The farmer reviews the order and either accepts or rejects it. If accepted, the stock is updated, and the buyer is notified of the final decision.



9. Database

- Users (Farmers, Buyers, Admins)
- Products (created by farmers)

- Orders (placed by buyers)
- Messages (chat system)
- Item (cart items)

The database will be **relational** because:

- Data has clear relationships
- Easy to manage structured data ● Perfect for marketplace systems

9.1.Core Entities

Users

Stores all system users:

- Farmer
- Buyer
- Admin

Products

Created by farmers and linked to them.

Orders

Connects:

Buyer ↔ Product ↔ Farmer

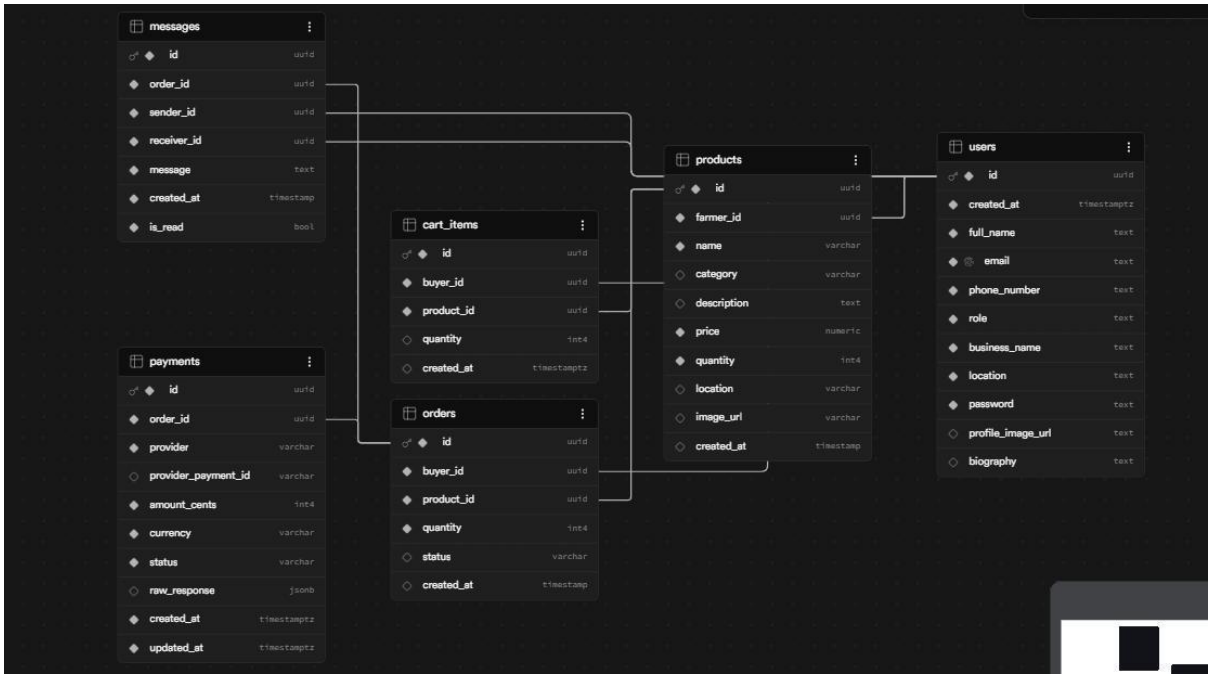
Messages

Used for chat between buyer and farmer.

Item

Used for storing cart items, and also for the order

9.2.ER-Digram



USERS		
int	id	PK
string	name	
string	email	
string	phone	
string	password	
string	role	
string	location	
datetime	created_at	

PRODUCTS		
int	id	PK
int	farmer_id	FK
string	name	
string	category	
string	description	
decimal	price	
int	quantity	
string	location	
string	image_url	
datetime	created_at	

ORDERS		
int	id	PK
int	buyer_id	FK
int	product_id	FK
int	quantity	
string	status	
datetime	created_at	

MESSAGES		
int	id	PK
int	order_id	FK
int	sender_id	FK
int	receiver_id	FK
string	message	
datetime	created_at	

9.3.Physical model

```
-- WARNING: This schema is for context only and is not meant to be run.
-- Table order and constraints may not be valid for execution.
CREATE TABLE public.cart_items ( id uuid
 NOT NULL DEFAULT gen_random_uuid(),
 buyer_id uuid NOT NULL, product_id uuid
 NOT NULL, quantity integer DEFAULT 1,
 created_at timestamp with time zone DEFAULT now(),
 CONSTRAINT cart_items_pkey PRIMARY KEY (id),
 CONSTRAINT fk_buyer FOREIGN KEY (buyer_id) REFERENCES public.users(id),
 CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES
 public.products(id)
);

CREATE TABLE public.messages ( id uuid NOT NULL
 DEFAULT gen_random_uuid(), order_id uuid NOT NULL,
 sender_id uuid NOT NULL, receiver_id uuid NOT NULL,
 message text NOT NULL DEFAULT 'NOT NULL'::text,
 created_at timestamp without time zone NOT NULL
 DEFAULT now(),
```

```
 is_read boolean NOT NULL DEFAULT false,
 CONSTRAINT messages_pkey PRIMARY KEY (id),
 CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES public.orders(id),
 CONSTRAINT fk_sender FOREIGN KEY (sender_id) REFERENCES public.users(id),
 CONSTRAINT fk_receiver FOREIGN KEY (receiver_id) REFERENCES public.users(id)
);
CREATE TABLE public.orders ( id uuid NOT NULL DEFAULT
 gen_random_uuid(), buyer_id uuid NOT NULL, product_id uuid NOT NULL,
 quantity integer NOT NULL, status character varying DEFAULT
 'pending'::character varying CHECK (status::text = ANY
(ARRAY['pending'::character varying, 'accepted'::character varying, 'rejected'::character
 varying]::text[])), created_at timestamp without time zone DEFAULT now(),
 CONSTRAINT orders_pkey PRIMARY KEY (id),
 CONSTRAINT fk_buyer FOREIGN KEY (buyer_id) REFERENCES public.users(id),
 CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES
 public.products(id)
);
CREATE TABLE public.products ( id uuid
 NOT NULL DEFAULT gen_random_uuid(),
 farmer_id uuid NOT NULL, name character
 varying NOT NULL, category character
 varying, description text,
 price numeric NOT NULL,
 quantity integer NOT NULL,
 location character varying,
 image_url character varying,
 created_at timestamp without time zone DEFAULT now(),
 CONSTRAINT products_pkey PRIMARY KEY (id),
 CONSTRAINT fk_farmer FOREIGN KEY (farmer_id) REFERENCES public.users(id),
 CONSTRAINT products_farmer_id_fkey FOREIGN KEY (farmer_id) REFERENCES
```

```
 public.users(id)
);
CREATE TABLE public.users ( id uuid NOT NULL
 DEFAULT gen_random_uuid(), created_at timestamp with
 time zone NOT NULL DEFAULT now(), full_name text
 NOT NULL, email text NOT NULL UNIQUE,
 phone_number text NOT NULL, role text NOT NULL,
 business_name text NOT NULL,
```

```
 location  text  NOT  NULL,
 password                text,
 profile_image_url       text,
 biography text,
 CONSTRAINT users_pkey PRIMARY KEY (id)
);

-- Payments table: store provider payment records tied to orders
CREATE  TABLE  public.payments  (  id  uuid  NOT  NULL  DEFAULT
 gen_random_uuid(), order_id uuid NOT NULL, provider character varying
 NOT NULL, provider_payment_id character varying, amount_cents integer
 NOT NULL, currency character varying(8) NOT NULL DEFAULT 'ETB',
 status  character  varying  NOT  NULL  DEFAULT  'pending'  CHECK
 (status::text = ANY
(ARRAY['pending'::character  varying,  'succeeded'::character  varying,  'failed'::character
varying,
'refunded'::character varying]::text[])),
 created_at timestamp with time zone NOT NULL DEFAULT now(),
 updated_at timestamp with time zone NOT NULL DEFAULT now(),
 CONSTRAINT payments_pkey PRIMARY KEY (id),
 CONSTRAINT  payments_order_id_fkey  FOREIGN  KEY  (order_id)  REFERENCES
 public.orders(id)
);
```

10. Structure of The App and UI/UX

AgriSpark App Prototype

Then inside, create these pages:

Main Screens Structure

- Auth Flow
    - Login

- Register

- Farmer Dashboard
  - Add/Edit Product
  - View Products
  - Accept/Reject Orders
  - Profile update
- Buyer Dashboard
  - search Product
  - see/browse details Products
  - Add to cart products
  - Placing order
  - Payment for the accepted orders
- Admin Dashboard
  - Managing products
  - Managing user [farmer | buyer]
  - Managing orders
  - See issues Report

10.1. Welcome / Landing Screen

Design:

- App name: **AgriSpark** ● Subtitle: "Connecting Farmers & Buyers" ● Buttons:
  - Login
  - Register

In Framer:

- Center layout
- Big heading ● Two buttons stacked

Landing Page UI

10.2 User Authentication

A. Login Screen

Fields:

- Email / Phone
- Password

Buttons:

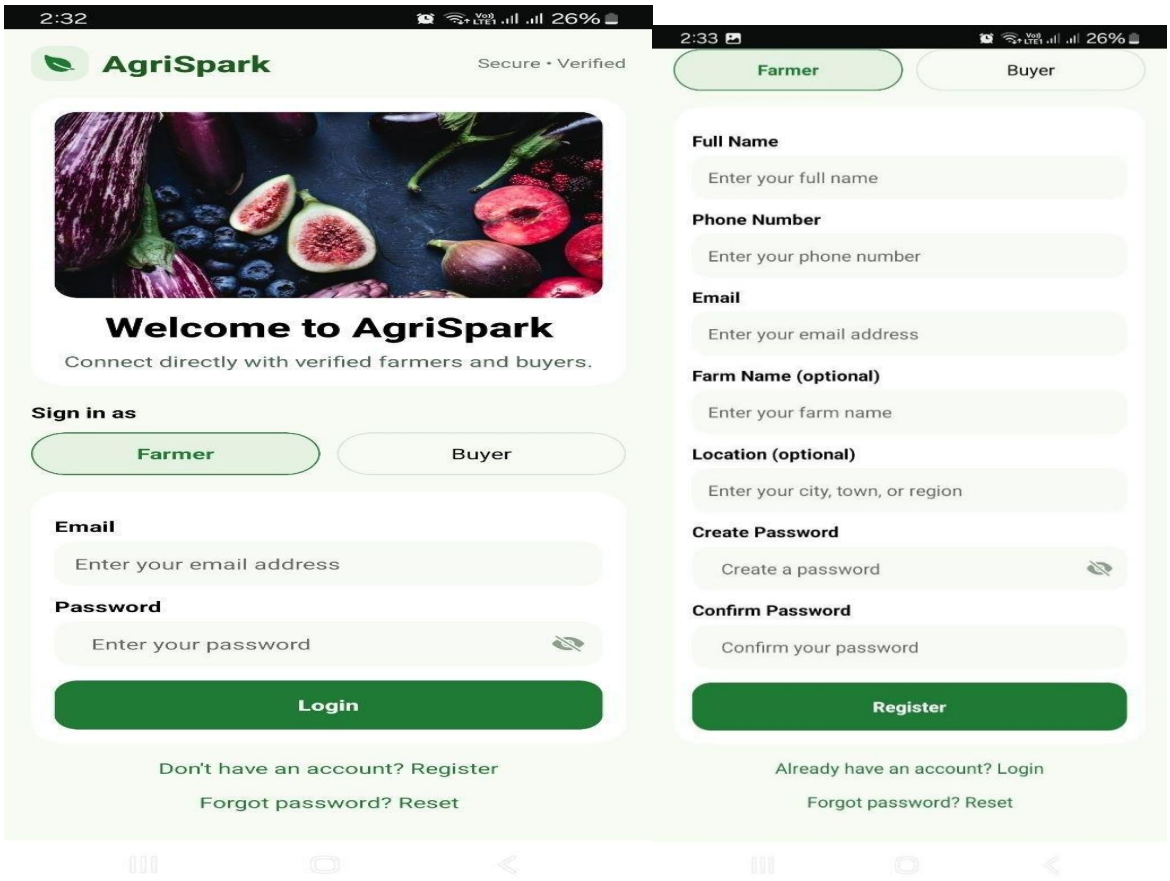
- Login
- "Don't have an account? Register"
- Password reset

B. Register Screen

Fields:

- Name
- Email/Phone
- Password
- Location

- Role selector:
    - Farmer
    - Buyer
    - admin



10.3. Farmer Screens

Farmer Dashboard

Sections:

- My Products
- Orders
- Add Product button

Cards:

- Product name
- Price
- Quantity ● Edit / Delete buttons

Add Product Screen

Fields:

- Product name
- Category
- Price
- Quantity ● Location
- Upload image

Button:

- Save Product

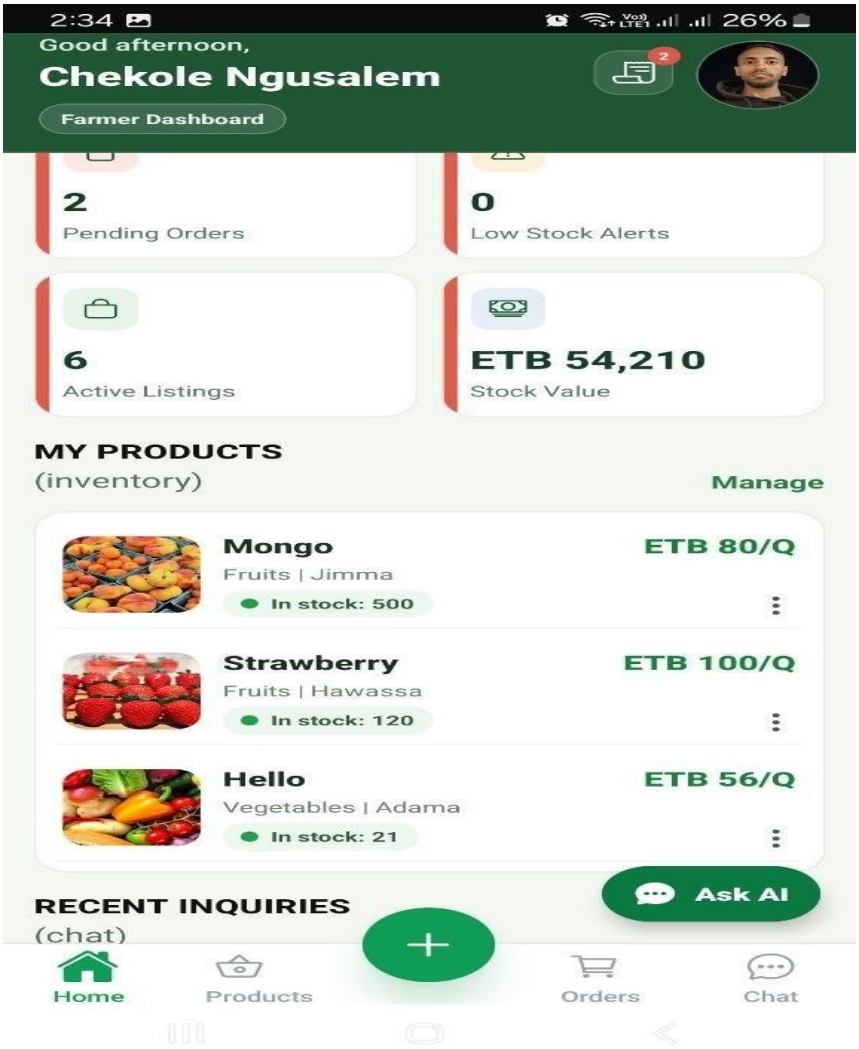
Farmer Orders Screen

Table/cards:

- Buyer name
- Product
- Quantity
- Status

Buttons:

- Accept
- Reject



10.4. Buyer Screens

Buyer Dashboard (Home)

Show:

- Product grid (cards) Each card:
- Image
- Name
- Price
- Location ● View Details button

Product Details Page

Show:

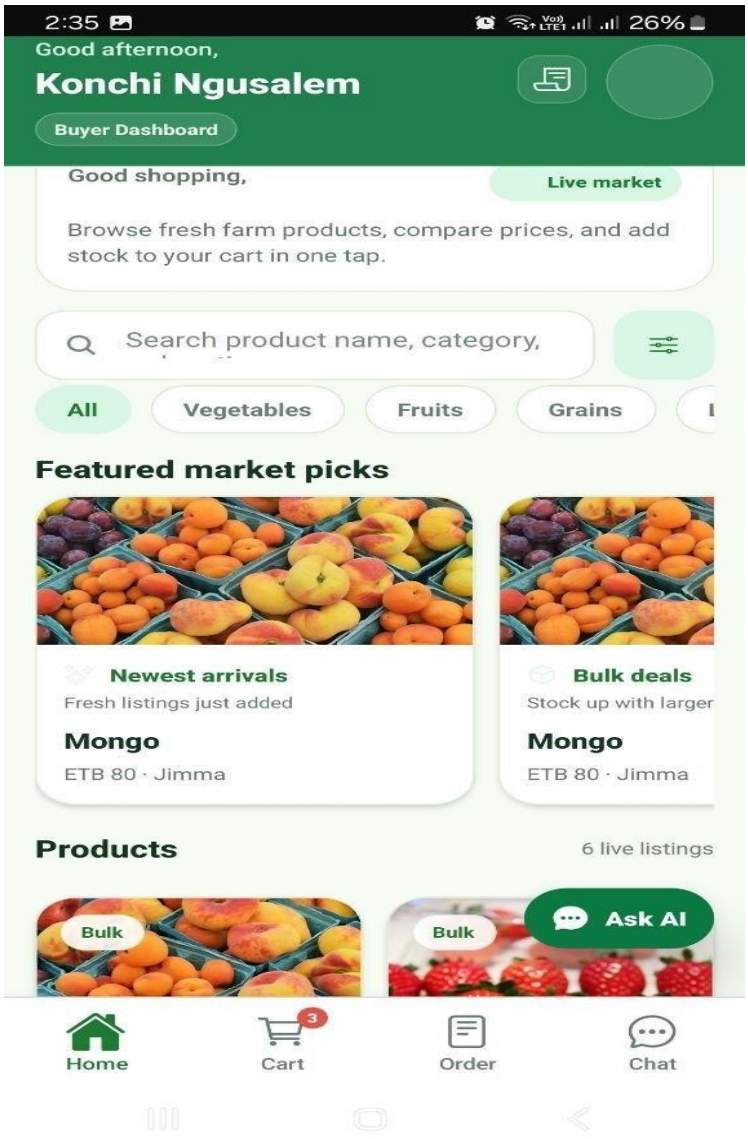
- Image
- Name

- Price
- Quantity available
- Farmer info
- Order quantity input Button:
- Place Order

Buyer Orders Page

List:

- Product
- Quantity
- Status (Pending / Accepted / Rejected)



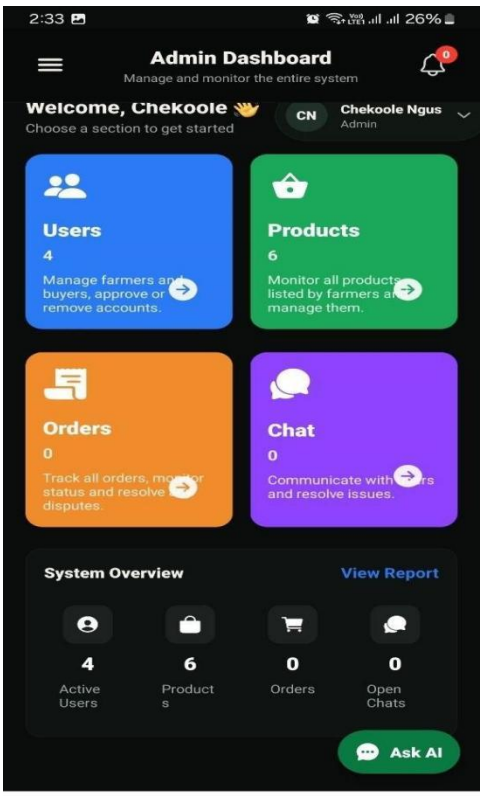
10.5. Admin Dashboard

Sections:

- Users
- Products
- Orders
- chat
- Analytics cards Example cards:
- Total Farmers
- Total Buyers
- Total Products

Buttons:

- Delete user
- Remove product



11. Development, Testing, and Deployment

A. Development

The AgriSpark mobile application was developed using modern web and mobile technologies.

Technologies Used

- ★ React Native (Expo)
- ★ JavaScript
- ★ Supabase Database & Authentication
- ★ PostgreSQL for Database inside Supabase ★ Git & GitHub for Version Control

Implemented Features

- ❖ User Authentication (Farmer, Buyer, Admin)
- ❖ Product Management
- ❖ Product Browsing
- ❖ Cart Management
- ❖ Order Management
- ❖ Real-Time Chat
- ❖ AI Assistant (AgriSpark AI)
- ❖ Multi-Language Support
- ❖ Admin Dashboard

B. Testing

The system was tested to ensure that all features work correctly and meet user requirements.

Functional Testing

Functional testing was conducted on all major system modules. The User Registration and Login module,

Product Management module, Product Browsing module, Cart Operations module, Order Placement module, Chat System, AI Assistant, and Admin Functions were tested and successfully passed all planned test cases. **Integration Testing**

- Front-end and backend communication tested successfully.
- Database operations validated.

- Authentication and authorization verified.

User Interface Testing

- Screens tested on different mobile devices.
- Navigation and responsiveness verified.

C. Deployment

The application was deployed using Expo for testing and distribution.

Deployment Process

- Application built using Expo.
- APK generated for Android testing.
- Source code managed through GitHub.
- Database hosted on Supabase.
- Back-end API deployed and connected to the mobile application.

Future Deployment

- Publish application to Google Play Store.
- Enable continuous updates and monitoring.

CONCLUSION

AgriSpark provides a comprehensive digital marketplace for Ethiopian agriculture. By connecting farmers and buyers directly, the platform reduces waste, increases profitability, improves communication, and promotes digital transformation within the agricultural sector. The system is scalable, secure, and designed for future expansion, making it a sustainable solution for modern agricultural commerce.